

LECTURE NOTES

Algorithm and Programming

Pointers and Array

Pandu Dwi Luhur Pambudi, S.Kom., M.Kom., M.I.M



Learning Outcomes

1. LO2: Menganalisis program untuk memecahkan masalah menggunakan bahasa C.

Outline Materi

1. Pointer
2. Pointer Constant & Pointer Variable
3. Array
4. String

5.1

Pointer

Pointer adalah suatu variabel yang berisi alamat memori dari suatu variabel lain. *Pointer* dilambangkan dengan operator “ * ” dan biasanya digunakan untuk memanipulasi isi variabel di dalam memori. Contoh penggunaannya adalah:

```
int a = 5;
```

```
int b = 3;
```

```
int *p;
```

```
p = &a; //1
```

```
b = *p; //2
```

Pada potongan program di atas dapat dilihat bahwa terdapat 3 buah variabel yaitu a, b dan pointer p. Variabel a bernilai 5, variabel b bernilai 3. Pada //1 p menerima alamat dari a, dalam hal ini operator “&” digunakan untuk mengambil alamat dari variabel a, jadi setelah statement //1 dijalankan maka pointer p akan memiliki alamat dari a. Untuk mengambil nilai dari alamat yang dipegang oleh pointer p dapat dilakukan dengan cara seperti statement //2. Dengan memanggil *p maka nilai dari variabel a (alamat yang dipegang oleh p) akan diambil, dari contoh di atas nilai tersebut diberikan ke variabel b. Seperti penggunaan array dalam sebuah variabel, sebuah pointer juga dapat menggunakan array. Seperti contoh berikut ini:

```
int *ap[10];
```

Dari contoh di atas dapat dilihat bahwa variabel ap adalah sebuah kumpulan variabel yang dapat menyimpan 10 alamat dari variabel lain. Index dari pointer tersebut dimulai dari *ap[0] sampai dengan *ap[9].

Contoh

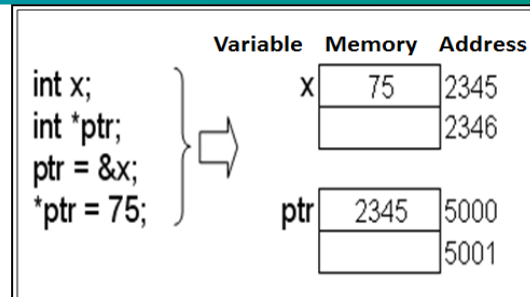
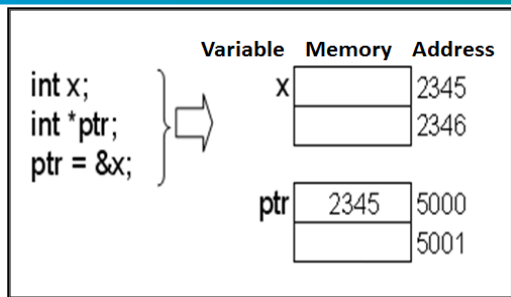
Inisialisasi pointer integer ke dalam variabel data:

```
int i, *ptr;
```

```
ptr = &i;
```

Untuk menetapkan nilai baru ke variabel yang ditunjuk oleh pointer:

```
*ptr = 5; / * berarti i = 5 */
```



Pointer to pointer adalah variabel yang menyimpan alamat lain dari pointer

Sintaks : `<type> **ptr_ptr ;`

Contoh

```
int i, *ptr, **ptr_ptr;
```

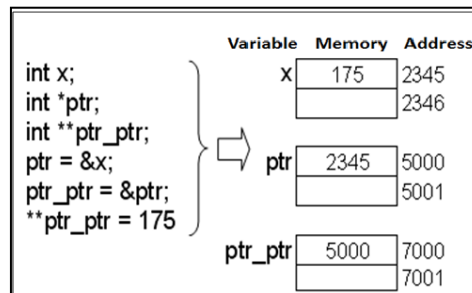
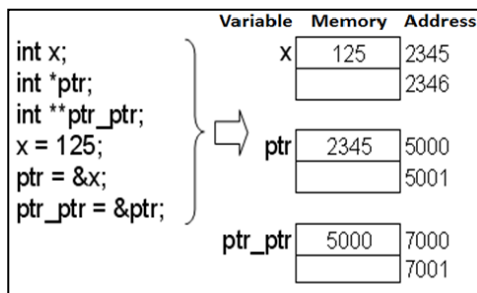
```
ptr = &i;
```

```
ptr_ptr = &ptr;
```

Untuk menetapkan nilai baru ke i:

```
*ptr = 5;           // means i=5 ;
```

```
**ptr_ptr = 9;      // means i=9; or *ptr=9;
```



Bahan Diskusi



Silahkan diskusikan bersama dosen/tutor/rekan mahasiswa mengenai:

Pertanyaan Diskusi:

Jelaskan peran penting pointer dalam bahasa C. Mengapa pointer dianggap sebagai salah satu fitur esensial tetapi juga sering membuat pemula bingung? Sertakan contoh situasi praktis di mana pointer sangat bermanfaat.

Hint:

- Pikirkan mengapa pointer penting untuk manajemen memori dinamis (`malloc`, `free`).
- Pertimbangkan bagaimana pointer mempermudah passing by reference, efisiensi manipulasi array, dan pembuatan struktur data kompleks (linked list, tree).
- Diskusikan juga kelemahannya: error segfault, dangling pointer.

Pertanyaan Diskusi:

Diskusikan bagaimana pointer dan array saling berkaitan dalam C. Apa persamaan dan perbedaannya? Berikan contoh bagaimana sebuah array dapat diakses menggunakan pointer, dan jelaskan potensi kesalahannya.

Hint:

- Ingat: array name adalah pointer ke elemen pertama.
- Bahas sintaks `*(arr + i)` sama dengan `arr[i]`.
- Diskusikan perbedaan: pointer bisa di-reassign, array name tidak bisa.
- Pikirkan risiko buffer overflow saat pointer keluar dari batas array.

Pertanyaan Diskusi:

Jelaskan apa itu pointer to pointer (`**ptr`) dan diskusikan contoh penggunaan praktisnya. Kapan kita butuh pointer ke pointer, dan apa tantangan yang muncul jika salah penanganan?

Hint:

- Pikirkan contoh: passing pointer by reference ke fungsi, array 2 dimensi, linked list dengan pointer ke pointer untuk insert/delete node.
- Diskusikan risiko: dereferencing pointer to pointer tanpa inisialisasi.
- Bahas sintaks: perbedaan `*ptr` dan `**ptr_ptr`.

Belajar Mandiri



Pelajari secara mandiri terkait dengan materi ini pada PPT dan video yang ada di LMS. Selain itu, rekan-rekan juga dapat menonton video berikut ini (C pointers explained: [Youtube](#)).

Pointer Constant & Pointer Variable

Dalam bahasa C, pointer merupakan salah satu fitur penting untuk mengakses dan memanipulasi data secara efisien di memori. Secara umum, pointer dapat dikategorikan menjadi **pointer constant** dan **pointer variable**. Keduanya memiliki perbedaan mendasar terkait bagaimana alamat yang mereka simpan dapat diubah selama program berjalan. Pemahaman konsep ini sangat penting agar programmer dapat menulis kode yang aman dan mudah dipelihara.

Pointer constant adalah pointer yang alamat memori yang dipegangnya **tidak dapat diubah** setelah ditetapkan pada saat kompilasi atau inisialisasi. Dengan kata lain, pointer jenis ini bersifat tetap menunjuk ke alamat yang sama selama program dieksekusi. Contoh paling umum dari pointer constant adalah **array**. Ketika mendeklarasikan array seperti `int B[4];`, nama `B` secara implisit adalah pointer constant yang selalu menunjuk ke alamat elemen pertama dari array tersebut.

Karena sifatnya yang konstan, pointer constant seperti array tidak bisa di-*assign* ke alamat memori lain. Misalnya, pernyataan `B = &x;` (dengan `x` sebagai variabel lain) tidak diperbolehkan dan akan menghasilkan error pada saat kompilasi. Hal ini memastikan bahwa struktur data array memiliki alamat dasar yang stabil, sehingga operasi akses elemen dengan indeks (`B[0]`, `B[1]`, dst.) tetap valid sepanjang waktu. Keamanan inilah yang membuat pointer constant sangat berguna untuk data yang strukturnya tidak berubah.

Sebaliknya, **pointer variable** adalah pointer yang **dapat diubah** alamat memori yang dipegangnya selama program berjalan (run-time). Pointer variable memungkinkan programmer untuk menunjuk ke berbagai lokasi memori yang berbeda sesuai kebutuhan. Contoh sederhana pointer variable adalah `int *ptr;`. Pointer `ptr` dapat disetel ke alamat variabel apa pun yang bertipe integer dengan menggunakan operator address-of (`&`). Misalnya, `ptr = &x;` dan kemudian `ptr = &y;` pada saat program dijalankan.

Perbedaan antara pointer constant dan pointer variable memberi fleksibilitas dan kontrol penuh dalam manajemen memori. Pointer variable banyak digunakan dalam pengalokasian memori dinamis (`malloc`), pembuatan struktur data kompleks seperti linked list atau tree, serta dalam mekanisme passing by reference ke fungsi. Di sisi lain, pointer constant seperti array memberikan kestabilan dan efisiensi dalam pengelolaan data berukuran tetap. Dengan memahami perbedaan ini, programmer dapat memilih tipe pointer yang tepat sesuai kebutuhan logika programnya.

Bahan Diskusi



Silahkan diskusikan bersama dosen/tutor/rekan mahasiswa mengenai:

Pertanyaan Diskusi:

Jelaskan bagaimana sifat stabil (alamat tetap) dari pointer constant seperti array dapat membantu dalam pengelolaan data dan keamanan program. Mengapa sifat ini penting pada operasi dengan data berukuran tetap?

Hint:

- Pikirkan bagaimana compiler bisa mengoptimasi akses ke array karena alamatnya tidak berubah.
- Diskusikan kasus di mana stabilitas ini mencegah error misalnya pointer pointing ke area memori salah.
- Coba hubungkan ke konteks stack dan memory layout.

Pertanyaan Diskusi:

Dalam situasi apa pointer variable lebih cocok digunakan daripada pointer constant? Berikan contoh praktis dalam konteks manajemen memori dinamis atau manipulasi struktur data.

Hint:

- Pikirkan penggunaan `malloc` atau `calloc` — hanya mungkin dengan pointer variable.
- Pertimbangkan struktur linked list, tree, atau graph — node perlu saling menunjuk secara fleksibel.
- Bahas passing by reference ke fungsi agar fungsi bisa memodifikasi variabel di luar scope-nya.

Pertanyaan Diskusi:

Diskusikan potensi risiko jika pointer variable digunakan secara sembarangan. Sebutkan contoh error umum (misalnya dangling pointer atau memory leak) dan bagaimana programmer dapat mencegahnya.

Hint:

- Pikirkan pointer pointing ke memori yang sudah dibebaskan (`free`).
- Bahas pentingnya inisialisasi pointer sebelum digunakan.
- Diskusikan pentingnya `NULL` pointer dan validasi sebelum dereferencing.

Belajar Mandiri



Pelajari secara mandiri terkait dengan materi ini pada PPT dan video yang ada di LMS. Selain itu, rekan-rekan juga dapat menonton video berikut ini (Constant Pointer VS. Pointer To A Constant: [Youtube](#)).

5.3

Array

Array adalah suatu tipe data terstruktur yang bertipe data sama dan berjumlah tetap (berdasarkan apa yang ditentukan) dan diberi suatu nama tertentu (sesuai variabel). Elemen-elemen array tersusun secara berurutan. Susunan tersebut membuat array memiliki alamat yang bersebelahan / berdampingan dalam memori sesuai dengan besar pemakaian memori tipe data yang digunakan. Namun, perlu diingat, walaupun elemen-elemen nya tersusun secara berurutan, array tetap dapat diakses secara acak di dalam memori. Array juga dapat berupa array 1 dimensi, 2 dimensi, bahkan n- dimensi.

Berikut contoh penggambaran array pada bahasa C.

0	1	2	3	4	5	6	7	indeks
10	44	2	7	25	56	32	40	value
1d2	1d4	1d6	1d8	1da	2dc	2de	1ed	alamat

Deklarasi Array :

```
data_type array_name[size];
```

contoh :

```
char nama[26];
```

Contoh diatas berarti anda memesan tempat di memori computer sebanyak 26 tempat dengan indeks yang dapat digunakan dari index 0 sampai index 25.

Array dapat diinisialisasi secara eksplisit tanpa deklarasi nilai dimensi

Contoh

```
int B[] = {1, 2, -4, 8};
```

Array B memiliki 4 elemen

1	2	-4	8
B[0]	B[1]	B[2]	B[3]

Contoh

```
int B[8] = {1, 2, -4, 8};
```

1	2	-4	8	0	0	0	0
B[0]	B[1]	B[2]	B[3]	B[4]	B[5]	B[6]	B[7]

Contoh

```
int B[4] = { 1, 2, -4, 8, 9 }; //error
```

kesalahan dalam hasil; nilai dimensi yang lebih kecil

Contoh inisialisasi array setelah definisi:

```
int A[5];
```

```
(for i=0; i<5;i++) A[i]=0;
```

```
int B[5];
B[5]={0,0,0,0,0};
```



Error, why ?

Contoh program array

```
#include <stdio.h>
void main()
{
    int i;
    int list_int[10];
    for (i=0; i<10; i++){
        list_int[i] = i + 1;
        printf( "list_int[%d] init with %d.\n", i, list_int[i]);
    }
}
```

a. Array 1 dimensi

Untuk memasukkan suatu nilai kedalam sebuah array, dapat dilakukan dengan beberapa cara, salah satu caranya adalah :

int list[5] = {2,1,3,7,8};

Contoh di atas berarti program akan memasukkan angka 2, 1, 3, 7 dan 8 kedalam sebuah array dengan nama list.

Contoh

```
#include<stdio.h>

int SIZE = 5;

void main() {
    int i, j;
    int n[SIZE] = {15, 9, 1, 7, 5};
    for( i=0 ; i<= SIZE ; i++) {
        printf("%5d ", n[i]);
        for ( j=1; j<=n[i] ; j++) printf("%c", "*");
        printf("\n");
    }
}
```

b. Array 2 dimensi

Sintaks : ***type name_array[row][col];***

Contoh

int a[3][4];

	Column 0	Column 1	Column 2	Column 3
Row 0	a[0][0]	a[0][1]	a[0][2]	a[0][3]
Row 1	a[1][0]	a[1][1]	a[1][2]	a[1][3]
Row 2	a[2][0]	a[2][1]	a[2][2]	a[2][3]

Array name Row subscript Column subscript

Contoh menggunakan rmo (row major order)

```

✓ int b[2][2] = {1, 2, 3, 4};
✓ int b[2][2] = { { 1, 2 }, { 3, 4 } };
✓ int b[2][2] = { { 1 }, { 3, 4 } };

✓ int x[3][4] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12};
✓ int x[3][4] = { {1, 2, 3, 4},
                  {5, 6, 7, 8},
                  {9, 10, 11, 12}
                };

```

1	2
3	4

1	0
3	4

Contoh program

```

/* Printing out array 2-D */
#include <stdio.h>
void main() {
    int two_dim[3][5] = {1, 2, 3, 4, 5,
                        10, 20, 30, 40, 50,
                        100, 200, 300, 400, 500};

    int i, j;
    for (i=0; i<3; i++){
        for (j=0; j<5; j++) printf("%6d", two_dim[i][j]);
        printf("\n");
    }
}

```

Output:

1 2 3 4 5

10 20 30 40 50

100 200 300 400 500

c. Array 3 dimensi

Untuk membuat array yang 3 dimensi (contohnya pada penggunaan array dalam konsep matrix).

Sintaks : type name_array[row][col][depth];

Dapat dilakukan inisialisasi dengan cara:

int variabel[x][y] = {

```

    {1, 2, ..., n},
    {1, 2, ..., n},
    ... dst
};

```

Dalam hal ini jumlah array yang dapat digunakan adalah mulai dari index `[0][0]` sampai dengan index `[x-1][y-1]`. Contoh:

```

int matrix[3][3] = {
    {11, 12, 13},
    {21, 22, 23},
    {32, 31, 33},
};

```

Contoh di atas berarti program akan memasukkan kumpulan angka yang berjumlah 9 tersebut kedalam array matrix yang akan menghasilkan: Matrix `[0][0] = 11` Matrix `[0][1] = 12` Matrix `[0][2] = 13` Matrix `[1][0] = 21` Matrix `[2][0] = 32` Matrix `[2][2] = 33` Dan seterusnya. Matrix yang dapat digunakan adalah mulai dari index `[0][0]` sampai dengan `[2][2]` yang bila dihitungkan berjumlah 9 buah.

Contoh

```

int x[3][2][4] = {{{1,2,3,4}, {5,6,7,8}},
                  {{11,12,13,14}, {15,16,17,18}},
                  {{21,22,23,24}, {25,26,27,28}}
};

void main() {
    int x[4][3][5] = {{{1, 2, 3}, {0, 4, 3, 4}, {1, 2}},
                      {{9, 7, 5}, {5, 7, 2}, {9}},
                      {{3, 3, 5}, {2, 8, 9, 9}, {1, 2, 1}},
                      {{0}, {1}, {0, 1, 9}}
    };

    printf("%5d", x[2][1][3]);
}

```

d. Array of Pointer

Array yang diisi dengan pointer/s

Sintaks : `type *array_name [value_dim];`

Contoh

```

int i;
int *ptr[4];
int x=1, y=2, z=3, w=5;
ptr[0]=&x, ptr[1]=&y, ptr[2]=&z, ptr[3]=&w;
for(i=0;i<4;i++) printf("%d ",*ptr[i]);

```

Output : 1 2 3 5

e. Array of Character

Array diisi dengan karakter/s

Sintaks : `char array_name[value_dim];`

Contoh

```
char name[40];
char ss[20]={ 'B','I','N','U','S'};           //20 elements
char ss[ ]= { 'B','I','N','U','S'};           // 5 elements
```

Bahan Diskusi



Setelah menyelesaikan materi ini, silahkan kerjakan soal berikut ini:

Topik:

Jelaskan bagaimana susunan elemen array di memori memengaruhi cara akses data pada array, serta bagaimana hal ini berbeda dengan tipe data pointer.

Hint:

- Diskusikan bahwa array memiliki elemen bersebelahan di memori → mendukung akses cepat.
- Bandingkan dengan pointer variable yang dapat menunjuk ke lokasi mana pun.
- Jelaskan bagaimana random access bekerja pada array meskipun tersusun berurutan.

Topik:

Berikan contoh skenario kesalahan dalam mendeklarasikan ukuran array dan inisialisasi nilainya. Mengapa kesalahan seperti `int B[4] = {1, 2, -4, 8, 9};` terjadi, dan bagaimana cara mencegahnya?

Hint:

- Bahas bahwa jumlah elemen inisialisasi tidak boleh melebihi ukuran yang dideklarasikan.
- Diskusikan cara inisialisasi tanpa perlu menyebutkan ukuran (otomatis dihitung).
- Hubungkan dengan praktik penulisan kode yang aman untuk mencegah runtime error.

Topik:

Bandingkan penggunaan array multidimensi (misalnya `int A[3][3]`) dengan array of pointer (`int *ptr[4]`). Dalam kasus apa masing-masing lebih sesuai digunakan?

Hint:

- Diskusikan bahwa array multidimensi cocok untuk data tabel/matriks tetap.
- Array of pointer fleksibel untuk menunjuk ke data yang lokasinya tersebar.
- Berikan contoh praktis, misalnya: array 2D untuk matriks matematika, array of pointer untuk daftar string atau array dinamis.

Belajar Mandiri



Pelajari secara mandiri terkait dengan materi ini pada PPT dan video yang ada di LMS. Selain itu, rekan-rekan juga dapat menonton video berikut ini (C arrays tutorial example explained: [Youtube](#)).

5.4

String

String adalah kumpulan karakter ASCII. Dalam bahasa C terdapat 2 cara mendeklarasikan variable string, satu sebagai pointer to character dan yang lain sebagai array of character. Penanda akhir string dilakukan dengan null character (,\'0\').

Berikut contoh array of character pada bahasa C:

```
char name[40] = "Amir"; //ok
```

Berikut contoh pointer to character pada bahasa C :

```
char label[] = "Single";
char label2[10] = "Married";
char *labelPtr;
labelPtr = label;
```

String adalah array of character dimana sebuah string ini dapat dimanipulasi layaknya sebuah array. Pada bahasa C menyediakan beberapa standart function yang dapat digunakan untuk manipulasi string. Function ini tersimpan pada header string.h.

Berikut beberapa fungsi string manipulasi yang ada pada bahasa C :

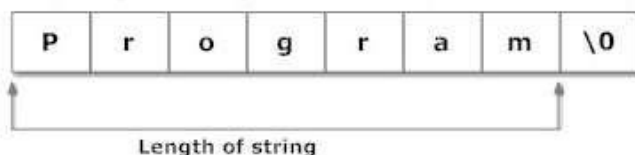
a. strlen()

Fungsi ini mengembalikan angka yang menyatakan panjangnya suatu string tanpa menghitung adanya null karakter pada ujung string.

Berikut adalah penggambaran dari fungsi strlen() pada bahasa C:

```
char c[]={'P','r','o','g','r','a','m','\0'};
temp=strlen(c);
```

Then, temp will be equal to 7 because, null character '\0' is not counted.



Berikut contoh penggunaan fungsi strlen() pada bahasa C :

```
#include <stdio.h>
#include <string.h>
int main(){
    char a[20]="Program";
    char b[20]={'P','r','o','g','r','a','m','\0'};
    char c[20];
    printf("Enter string: ");
    gets(c);
    printf("Length of string a=%d \n",strlen(a));
    //calculates the length of string before null charcter.
    printf("Length of string b=%d \n",strlen(b));
    printf("Length of string c=%d \n",strlen(c));
    return 0;
}
```

Pada program tersebut terlihat bahwa fungsi `strlen()` pada C menghitung banyaknya kata pada suatu variable. Jika dimasukkan kata „String” pada program tersebut maka program tersebut akan mengeluarkan panjang karakter dari variable a, b, dan c. Sehingga dari program tersebut akan menghasilkan output sebagai berikut :

```
Enter string: String
Length of string a=7
Length of string b=7
Length of string c=6
```

b. `strcpy(s1,s2)`

Fungsi ini menyalin string dari s2 ke s1, s2 dapat berupa variable atau string constant. Berikut adalah contoh penggunaan fungsi `strcpy()` pada bahasa C:

```
#include <stdio.h>
#include <string.h>
int main(){
    char a[10],b[10];
    printf("Enter string: ");
    gets(a);
    strcpy(b,a);    //Content of string a is copied to string b.
    printf("Copied string: ");
    puts(b);
    return 0;
}
```

Jika pada program tersebut dimasukkan kata “Programming Tutorial” yang kemudian akan disimpan pada variabel a. Kemudian digunakan fungsi `strcpy()` dimana variabel a disalin ke variabel b. Sehingga akan menghasilkan output sebagai berikut :

```
Copyed string: Programming Tutorial    \\ASCIIcode p
Enter string: Programming Tutorial    \\ASCIIcode g
```

c. `strncpy(s1,s2,n)`

Fungsi ini menyalin string dari s2 ke s1 sebanyak n karakter, s2 dapat berupa variable atau string constant. Berikut adalah contoh penggunaan fungsi `strncpy()` pada bahasa C :

```
#include <stdio.h>
#include <string.h>

int main()
{
    char src[40];
    char dest[12];

    memset(dest, '\0', sizeof(dest));
    strcpy(src, "This is tutorialspoint.com");
    strncpy(dest, src, 10);

    printf("Final copied string : %s\n", dest);

    return(0);
}
```

Pada program tersebut variable src memiliki nilai „This is tutorialspoint.com” dimana nilai dari variable src ini disalin ke variabel dest. Penyalinan dilakukan hanya sebanyak 10 karakter sehingga hasil output dari program tersebut adalah :

```
Final copied string : This is tu
```

d. `strcat(s1,s2)`

Fungsi ini berguna untuk menambahkan nilai string dari s2 keakhir string s1, s2 dapat berupa variable atau string constant. Berikut adalah contoh penggunaan fungsi `strcat()` pada bahasa C :

```
#include <stdio.h>
#include <string.h>

int main ()
{
    char src[50], dest[50];

    strcpy(src, "This is source");
    strcpy(dest, "This is destination");

    strcat(dest, src);

    printf("Final destination string : |%s|", dest);

    return(0);
}
```

Pada program tersebut variable `src` memiliki nilai „This is source“ dan variable `dest` memiliki nilai „This is destination“. Jika menggunakan fungsi `strcat` maka variable `src` akan ditambahkan diakhir variable `dest`. Sehingga hasil output dari program tersebut adalah :

```
Final destination string : |This is destinationThis is source|
```

e. `strncat(s1,s2,n)`

Fungsi ini berguna untuk menambahkan n karakter pada string dari s2 keakhir string s1, s2 dapat berupa variable atau *string constant*. Berikut adalah contoh penggunaan fungsi `strncat()` pada bahasa C :

```
#include <stdio.h>
#include <string.h>

int main ()
{
    char src[50], dest[50];

    strcpy(src, "This is source");
    strcpy(dest, "This is destination");

    strncat(dest, src, 15);

    printf("Final destination string : |%s|", dest);

    return(0);
}
```

Pada program tersebut variable `src` memiliki nilai „This is source“ dan variable `dest` memiliki nilai „This is destination“. Jika menggunakan fungsi `strncat` maka variable `src` akan ditambahkan sebanyak 15 karakter diakhir variable `dest`. Sehingga hasil output dari program tersebut adalah :

```
Final destination string : |This is destinationThis is source|
```

f. strcmp(s1,s2)

Fungsi ini berguna untuk membandingkan antara string s1 dan string s2, secara alfabetis. Berikut adalah contoh penggunaan fungsi strcmp() pada bahasa C :

```
#include <stdio.h>
#include <string.h>

int main ()
{
    char str1[15];
    char str2[15];
    int ret;

    strcpy(str1, "abcdef");
    strcpy(str2, "ABCDEF");

    ret = strcmp(str1, str2);

    if(ret < 0)
    {
        printf("str1 is less than str2");
    }
    else if(ret > 0)
    {
        printf("str2 is less than str1");
    }
    else
    {
        printf("str1 is equal to str2");
    }

    return(0);
}
```

Pada program tersebut dilakukan perbandingan antara 2 variabel yaitu str1 "abcdef" dengan variabel str2 "ABCDEF". Terdapat variabel **ret** yang berguna untuk mengembalikan nilai dari fungsi *strcmp()* ini. Dengan ketentuan nilai fungsi yang dikembalikan sebagai berikut :

- jika nilai kembali < 0 maka str1 memiliki nilai kurang dari str2.
- Jika nilai kembali > 0 maka str2 memiliki nilai kurang dari str1.
- Jika nilai kembali = 0 maka str1 memiliki nilai yang sama dengan str2.

Sehingga hasil output dari program tersebut adalah :

```
str2 is less than str1
```

Bahan Diskusi



Setelah menyelesaikan materi ini, silahkan kerjakan soal berikut ini:

Pertanyaan Diskusi:

Jelaskan perbedaan konsep dan perilaku antara string yang dideklarasikan sebagai array of character dan string yang dideklarasikan sebagai pointer to character. Kapan sebaiknya menggunakan masing-masing cara?

Hint:

- Ingat: `char name[40]` menyimpan ruang memori tetap, sedangkan `char *label` hanya pointer yang bisa diarahkan ke string literal atau buffer lain.
- Bahas keterbatasan pointer to literal: string literal di segmen read-only.
- Diskusikan kapan array lebih aman (untuk modifikasi) dan pointer lebih efisien (untuk konstanta).

Pertanyaan Diskusi:

Diskusikan bagaimana fungsi manipulasi string seperti `strcpy()`, `strncpy()`, `strcat()`, dan `strcmp()` bekerja. Apa risiko yang bisa muncul jika programmer tidak berhati-hati, terutama terkait buffer overflow?

Hint:

- Ingat: `strcpy()` dan `strcat()` tidak mengecek panjang buffer tujuan.
- Diskusikan contoh nyata: penyalinan string lebih panjang dari buffer, menyebabkan overwrite memori.
- Coba sebutkan solusi: gunakan `strncpy()` atau validasi panjang string dengan `strlen()` sebelum salin/tambah.

Pertanyaan Diskusi:

Jelaskan peran null character (`\0`) pada string dalam bahasa C. Apa yang bisa terjadi jika string tidak diakhiri dengan null character? Bagaimana fungsi seperti `strlen()` dan `strcpy()` terpengaruh?

Hint:

- Bahas cara kerja fungsi string: fungsi iterasi hingga `\0` ditemukan.
- Diskusikan efek tidak ada null terminator: pembacaan memori berlebih, garbage value, atau crash.
- Tanyakan: bagaimana programmer memastikan string selalu valid?

Belajar Mandiri



Pelajari secara mandiri terkait dengan materi ini pada PPT dan video yang ada di LMS. Selain itu, rekan-rekan juga dapat menonton video berikut ini (String Basics: [Youtube](#)).

Post-Test

Setelah menyelesaikan materi ini, silahkan kerjakan soal berikut ini:

Soal 1 (Pointer)

Perhatikan potongan kode berikut:

```
int x = 10;
int *p;
p = &x;
*p = 20;
printf("%d", x);
```

Apa output dari program di atas?

- A. 10
- B. 20
- C. Alamat variabel x
- D. Error kompilasi

Soal 2 (Pointer Constant vs Pointer Variable)

Manakah pernyataan yang benar mengenai pointer constant?

- A. Alamat yang disimpan dapat diubah di run-time.
- B. Pointer constant dapat digunakan sebagai array.
- C. Pointer constant hanya digunakan pada fungsi.
- D. Pointer constant tidak dapat menunjuk ke array.

Soal 3 (Array)

Diberikan deklarasi berikut:

```
int arr[5] = {1, 2, 3, 4, 5};
printf("%d", *(arr + 2));
```

Apa output yang dicetak?

- A. 1
- B. 2
- C. 3
- D. 4

Post-Test



Setelah menyelesaikan materi ini, silahkan kerjakan soal berikut ini:

Soal 4 (Array 2 Dimensi)

Perhatikan deklarasi array 2 dimensi berikut:

```
int mat[2][3] = { {1, 2, 3}, {4, 5, 6} };  
printf("%d", mat[1][2]);
```

Apa hasil outputnya?

- A. 2
- B. 3
- C. 4
- D. 6

Soal 5 (String)

Apa hasil output dari program berikut?

```
char str1[20] = "Hello";  
char str2[20];  
strcpy(str2, str1);  
printf("%s", str2);
```

- A. Hello
- B. str1
- C. Error
- D. Null

Kesimpulan

1. Pointer adalah suatu variabel yang berisi alamat memori dari suatu variabel lain. Pointer dilambangkan dengan operator “*” dan biasanya digunakan untuk memanipulasi isi variabel di dalam memori.
2. Pointer constant adalah pointer yang tidak dapat ditetapkan dengan nilai baru pada saat run-time. Pointer variabel adalah pointer yang diberikan dengan nilai baru pada saat run-time.
3. Array adalah suatu tipe data terstruktur yang bertipe data sama dan berjumlah tetap (berdasarkan apa yang ditentukan) dan diberi suatu nama tertentu (sesuai variabel).
4. String adalah kumpulan karakter ASCII. Dalam bahasa C terdapat 2 cara mendeklarasikan variable string, satu sebagai pointer to character dan yang lain sebagai array of character. Terdapat fungsi string manipulasi yang dapat digunakan dalam Bahasa C:
 - `strlen()`
 - `strcpy(s1,s2)`
 - `strncpy(s1,s2,n)`
 - `strcat(s1,s2)`
 - `strncat(s1,s2,n)`
 - `strcmp(s1,s2).`

Daftar Pustaka

- Paul J. Deitel & Harvey. Deitel. (2022). C how to program. 09. Pearson Education. Hoboken. ISBN:978-0-13-739839-3. Chapter 7 & 8
- C Programming – Pointers: <http://www.exforsys.com/tutorials/c-language/c-pointers.html>
- <https://www.cs.bu.edu/teaching/cpp/string/array-vs-ptr/>
- <http://www.programiz.com/c-programming/library-function/string.h>
- http://www.tutorialspoint.com/c_standard_library/string_h.htm
- Storing Similar Data Items: <http://aelinik.free.fr/c/ch12.htm>

Berikut jawaban untuk pertanyaan Post-Test diatas:

1. **Jawaban: B**

Alasan: `*p = 20;` mengubah nilai `x` melalui pointer, sehingga `x` berubah menjadi 20.

2. **Jawaban: B**

Alasan: Array adalah contoh pointer constant; nama array adalah pointer constant yang tidak dapat diubah.

3. **Jawaban: C**

Alasan: `*(arr + 2)` mengakses elemen ke-3 (indeks 2) dari array, yaitu 3.

4. **Jawaban: D**

Alasan: `mat[1][2]` berarti baris ke-2 (indeks 1), kolom ke-3 (indeks 2), nilainya adalah 6.

5. **Jawaban: A**

Alasan: `strcpy` menyalin string "Hello" dari `str1` ke `str2`; output `str2` adalah "Hello".